

Proyecto integrador EDD - 20261

Mini motor documental basado en árboles y listas enlazadas

Objetivo

Diseñar e implementar un sistema básico de almacenamiento y consulta de documentos tipo JSON, inspirado en el funcionamiento general de una colección documental como MongoDB.

El sistema debe recibir un JSON que representa varios documentos de una misma colección, convertir cada documento en un árbol propio, almacenar la colección como una lista enlazada de documentos y permitir consultas básicas sobre los datos.

Contexto

El JSON de entrada representa una colección de documentos:

```
[
  {
    "id": 1,
    "nombre": "Ana",
    "edad": 25,
    "ciudad": "Medellín",
    "direccion": {
      "barrio": "Laureles",
      "codigo_postal": 50031
    }
  },
  {
    "id": 2,
    "nombre": "Carlos",
    "edad": 31,
    "ciudad": "Bogotá",
    "direccion": {
      "barrio": "Chapinero",
      "codigo_postal": 110231
    }
  }
]
```

Cada documento debe transformarse en una estructura de árbol.

Ejemplo conceptual:

```
documento
├── id: 1
├── nombre: "Ana"
├── edad: 25
├── ciudad: "Medellín"
└── direccion
    ├── barrio: "Laureles"
    └── codigo_postal: 50031
```

La colección completa debe almacenarse como una lista enlazada:

doc_1 → doc_2 → doc_3 → ... → None

Cada nodo de la lista enlazada apunta al árbol que representa un documento.

Requisitos funcionales

1. Carga de colección

El sistema debe permitir recibir una colección en formato JSON.

Puede recibirse como:

- un archivo `.json`.

Cada documento de la colección debe convertirse en un árbol.

2. Representación con árboles

- Cada documento debe representarse como un árbol.
 - Los hijos de cada nodo **no deben almacenarse usando listas de Python**. Deben almacenarse usando una **lista enlazada propia**.
-

3. Representación de la colección

La colección completa debe almacenarse usando una **lista enlazada propia**.

Cada nodo de la lista enlazada debe contener una referencia a la raíz del árbol de un documento.

No se permite usar listas de Python para representar internamente la colección.

4. Conversión de JSON a árbol

El sistema debe convertir cada documento JSON en un árbol.

Debe soportar, como mínimo:

strings
números
booleanos
null
objetos anidados

5. Conversión de árbol a JSON

El sistema debe poder reconstruir el JSON original desde los árboles almacenados.

Es decir, después de cargar la colección y convertirla a árboles, debe ser posible volver a obtener una estructura equivalente al JSON original.

6. Consulta básica tipo MongoDB

El sistema debe permitir consultar documentos usando filtros simples.

Ejemplos:

```
collection.find({"ciudad": "Medellín"})
```

Debe retornar todos los documentos cuya ciudad sea Medellín.

También debe soportar rutas anidadas:

```
collection.find({"direccion.barrio": "Laureles"})
```

7. Operadores de consulta

El sistema debe soportar los siguientes operadores:

\$eq igual que
\$ne diferente de
\$gt mayor que
\$gte mayor o igual que
\$lt menor que
\$lte menor o igual que

Ejemplos:

```
collection.find({"edad": {"$gt": 25}})
```

```
collection.find({"edad": {"$gte": 18, "$lte": 30}})
```

```
collection.find({"ciudad": {"$ne": "Bogotá"}})
```

8. Consultas con múltiples condiciones

El sistema debe permitir consultas con más de una condición.

Ejemplo:

```
collection.find({  
  "ciudad": "Medellín",  
  "edad": {"$gte": 18}  
})
```

La consulta anterior debe retornar únicamente los documentos que cumplan **todas** las condiciones.

Restricciones obligatorias

1. No se permite usar listas de Python para representar:
 - la colección;

- los hijos de un nodo del árbol.
 - 2. No se permite usar diccionarios de Python como estructura interna principal después de convertir el JSON a árbol.
 - 3. Sí se permite usar diccionarios temporalmente para:
 - leer el JSON inicial;
 - representar la consulta;
 - reconstruir el JSON de salida.
 - 4. Se deben implementar clases propias para:
 - nodo del árbol;
 - lista enlazada de hijos;
 - nodo de la colección;
 - lista enlazada de documentos;
 - colección documental.
 - 5. El sistema debe incluir type hints.
 - 6. El sistema debe manejar casos en los que:
 - la ruta consultada no existe;
 - el tipo de dato no permite aplicar un operador;
 - la colección está vacía;
 - no hay documentos que cumplan la consulta.
-

Ejemplo de uso esperado

```
data = [  
    {  
        "id": 1,  
        "nombre": "Ana",  
        "edad": 25,  
        "ciudad": "Medellín",  
        "direccion": {  
            "barrio": "Laureles"  
        }  
    },  
    {  
        "id": 2,  
        "nombre": "Carlos",  
        "edad": 31,  
        "ciudad": "Bogotá",  
        "direccion": {  
            "barrio": "Chapinero"  
        }  
    }  
]
```

```
collection = DocumentCollection()
```

```
collection.load(data)

print(collection.find({"ciudad": "Medellín"}))
print(collection.find({"edad": {"$gt": 25}}))
print(collection.find({"direccion.barrio": "Laureles"}))

print(collection.to_json())
```

Entregables

1. Código fuente completo.
 2. Archivo o celda con ejemplos de prueba.
 3. Breve explicación del modelo de clases.
 4. Análisis de complejidad de:
 - carga de documentos;
 - búsqueda por ruta;
 - consulta sobre toda la colección;
 - conversión de árbol a JSON.
 5. Evidencia de ejecución con al menos 5 consultas diferentes.
-

Criterios de evaluación implementación (40%)

Criterio	Porcentaje
Conversión JSON → árbol	20%
Uso correcto de listas enlazadas propias	20%
Conversión árbol → JSON	20%
Consultas simples y anidadas	25%
Operadores de comparación	10%
Manejo de casos borde	5%
Sustentación → 60% restante de la nota completa	
